

Code Intelligence Agent

High-Level Design Document

Code-Intelligence-Agent / Design Team

February 2026

Abstract

This document provides a high-level architectural overview of the **Code Intelligence Agent**, an AI-powered intelligent coding assistant designed for students, trainers, and working professionals. The system integrates a seven-stage intelligent pipeline — Adaptive Explanation, Code Generation, Code Explanation, Debugging, Optimality Score Check, Visual Flow Explanation, and Descriptive Explanation Mode — using a React + Tailwind CSS frontend, a Python FastAPI backend, Hugging Face Transformers for local LLM inference, and a Mermaid + Kroki pipeline for diagram rendering. This document covers system architecture, component interactions, data flow, security design, key feature workflows, and deployment architecture without implementation-level code details.

Contents

1	System Overview	1
1.1	Platform Vision	1
1.2	Core Capabilities	1
1.3	Technology Philosophy	1
2	Architectural Blueprint	2
2.1	High-Level System Architecture	2
2.1.1	Architectural Components	2
2.1.2	Communication Patterns	3
2.2	Data Flow Architecture	3
2.2.1	Key Data Pathways	3
2.2.2	Consistency Model	4
3	Component Architecture	4
3.1	Frontend Component Structure	4
3.1.1	Architectural Layers	5
3.2	UI/UX Design Strategy	5
3.2.1	Design System Principles	5
3.2.2	User Experience Patterns	6
4	Backend Architecture	6
4.1	Service Architecture	6
4.1.1	Service Components	6
4.2	Data Model Design	6
4.2.1	Core Entities	6
4.2.2	Relationship Patterns	8
5	Agentic Pipeline Architecture	8
5.1	Seven-Stage Pipeline Strategy	8
5.1.1	Pipeline Stage Details	9
5.1.2	Pipeline Reliability Strategy	10
6	Request–Response Communication Architecture	10
6.1	HTTP Pipeline Communication Strategy	10
6.1.1	Communication Patterns	11
6.1.2	Event Management	12
6.2	Consistency and Reliability	12
7	Security Architecture	12
7.1	Security Model	12
7.1.1	Security Strategy	13
7.1.2	Security Measures	13

8	Key Feature Workflows	13
8.1	Code Intelligence Workflow	13
8.1.1	User Interaction Flow	14
8.1.2	Technical Implementation Approach	14
8.2	AI Voice / Text Integration	15
8.2.1	AI Integration Strategy	15
9	System Integration	15
9.1	External Service Integration	15
9.2	API Design Principles	15
9.3	Deployment Architecture	16
10	Performance and Scalability	16
10.1	Performance Characteristics	16
10.2	Scalability Strategy	16
11	Conclusion	17
11.1	Architectural Strengths	17
11.2	Implementation Readiness	17
11.3	Future Considerations	17

1 System Overview

1.1 Platform Vision

The Code Intelligence Agent is a modern, AI-enhanced programming assistance platform enabling students, trainers, and working professionals to generate, understand, debug, optimise, and visualise source code efficiently. The system combines locally hosted large language model inference with an adaptive explanation engine that calibrates output depth and vocabulary to the user's declared knowledge level (Beginner, Intermediate, or Advanced), making intelligent programming assistance genuinely accessible without prior AI expertise.

1.2 Core Capabilities

- **Adaptive Explanation System:** Detects the user's knowledge level and automatically adjusts explanation depth, vocabulary, and structure across all operations.
- **Code Generation:** Accepts natural language prompts and generates complete, executable source code in Python, Java, C, C++, and JavaScript using the Hugging Face LLM.
- **Code Explanation:** Produces line-by-line, function-level, and concept-mapped explanations of submitted source code, calibrated to the user's selected level.
- **Debugging System:** Analyses submitted code to detect syntax errors, logical bugs, and root causes, returning corrected and optimised code with issue explanations.
- **Optimality Score Check:** Evaluates code quality with a numerical score (0–100) using AST analysis and LLM-based quality assessment, with ranked improvement suggestions.
- **Visual Flow Explanation:** Converts source code into Mermaid diagram syntax (flowcharts, sequence, class diagrams) rendered via the Kroki API into SVG or PNG.
- **Descriptive Explanation Mode:** Provides high-level theory explanations and concept simplification without requiring code submission, targeted at students.

1.3 Technology Philosophy

The system adopts a modular, React-first full-stack architecture with emphasis on adaptive intelligence, security, and educational accessibility:

- **Adaptive Intelligence:** Knowledge-level context is stored in session state and injected automatically into every AI prompt template.
- **Separation of Concerns:** Each capability is handled by a dedicated FastAPI router with clearly defined JSON interfaces.
- **Security by Design:** User code is never executed on the server; all AI inference is

local; external calls use HTTPS.

- **Scalability:** FastAPI's async architecture supports concurrent request handling and horizontal scaling with minimal structural changes.

2 Architectural Blueprint

2.1 High-Level System Architecture

The system follows a three-tier architecture connecting the React + Tailwind CSS User Interface Layer, the Python FastAPI Service Logic Layer, and the Hugging Face AI and Kroki Integration Layer via REST APIs and HTTPS.

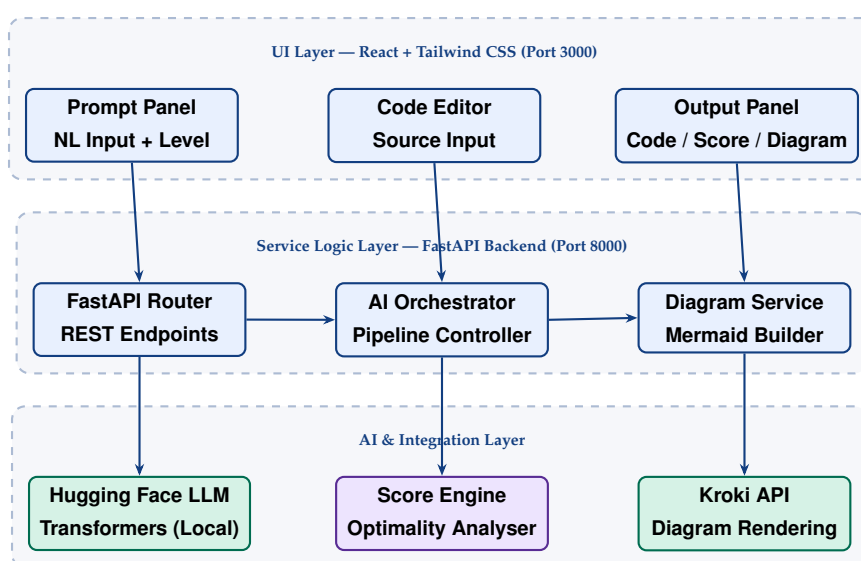


Figure 1: High-Level System Architecture Overview

2.1.1 Architectural Components

1. **Client Layer — React + Tailwind CSS Frontend:** The SPA provides a prompt panel with knowledge-level selector, a code editor, and a structured output panel. All backend communication uses Axios REST calls with Tailwind-styled loading states.
2. **Application Layer — FastAPI Backend:** Exposes async REST endpoints per operation, validates all inputs, routes to service modules, and assembles JSON responses with automatic OpenAPI documentation at /docs.
3. **AI Processing Layer — Hugging Face Transformers:** Hosts the LLM locally via the pipeline() interface; the model is loaded once at startup. Knowledge-level context is injected into every prompt template.
4. **Score Engine — Optimality Analyser:** Combines Python AST static analysis with LLM-based quality assessment to produce a 0–100 score and ranked suggestions.
5. **Diagram Rendering Layer — Mermaid + Kroki:** MermaidBuilder produces valid

diagram syntax; KrokiClient submits it over HTTPS for SVG/PNG rendering.

2.1.2 Communication Patterns

- **RESTful JSON API (HTTP):** Axios POST calls from frontend to FastAPI endpoints; each endpoint handles one operation type.
- **HTTPS External Calls:** Mermaid syntax is submitted to Kroki over HTTPS with TLS; no user code or credentials are included.
- **Local Function Calls for AI:** FastAPI calls the Hugging Face runtime directly via Python function calls, keeping all inference within the local process.

2.2 Data Flow Architecture

The system implements a sequential, stage-gated data flow where each stage receives validated input from the previous and failures are caught and reported without crashing subsequent stages.

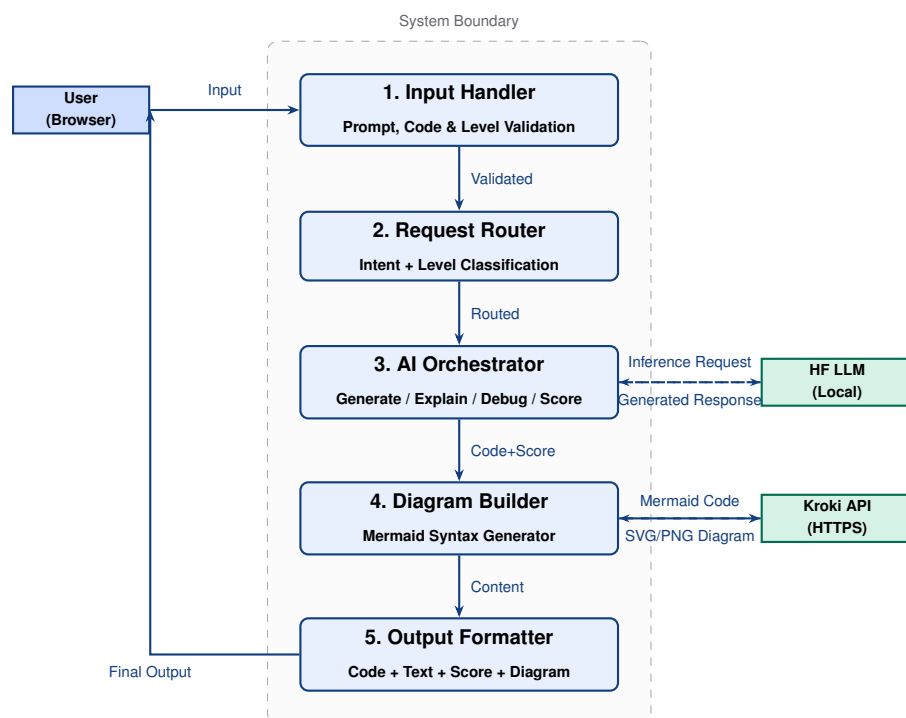


Figure 2: System Data Flow Diagram

2.2.1 Key Data Pathways

1. **Input Validation Flow:** Every request passes through the Input Handler, which enforces non-empty content, length limits, injection-pattern checks, and a valid knowledge-level field before forwarding.
2. **Adaptive Code Generation Flow:** The Router classifies the intent and injects the knowledge level into the LLM prompt template; the Orchestrator parses the code

block response.

3. **Code Explanation Flow:** Source code is routed to the Orchestrator with a level-aware prompt producing line-by-line breakdown, concept mapping, and logical flow.
4. **Debugging Flow:** The Orchestrator builds a prompt requesting all error identifications, root causes, and a corrected code version.
5. **Optimality Score Flow:** The Score Engine performs AST analysis and LLM quality assessment, returning a JSON ScoreReport with numeric rating and ranked suggestions.
6. **Visualisation Flow:** The Diagram Builder constructs Mermaid syntax, validates it locally, submits to Kroki, and returns the rendered SVG to the Output Formatter.

2.2.2 Consistency Model

- **Stage-Gated Sequential Pipeline:** Each stage completes fully before the next is invoked; failures halt downstream stages immediately.
- **Knowledge-Level Propagation:** The user's level is attached to the validated request and carried through all pipeline stages automatically.
- **Stateless API Requests:** Each FastAPI call carries all required data in its payload; no shared mutable state exists between requests.
- **Graceful Error Recovery:** Kroki failure returns raw Mermaid syntax as fallback; partial results are always preferred over total failure.

3 Component Architecture

3.1 Frontend Component Structure

The frontend follows a layered React component architecture using Tailwind CSS, with strict separation between infrastructure, global state, layout, and feature pages.

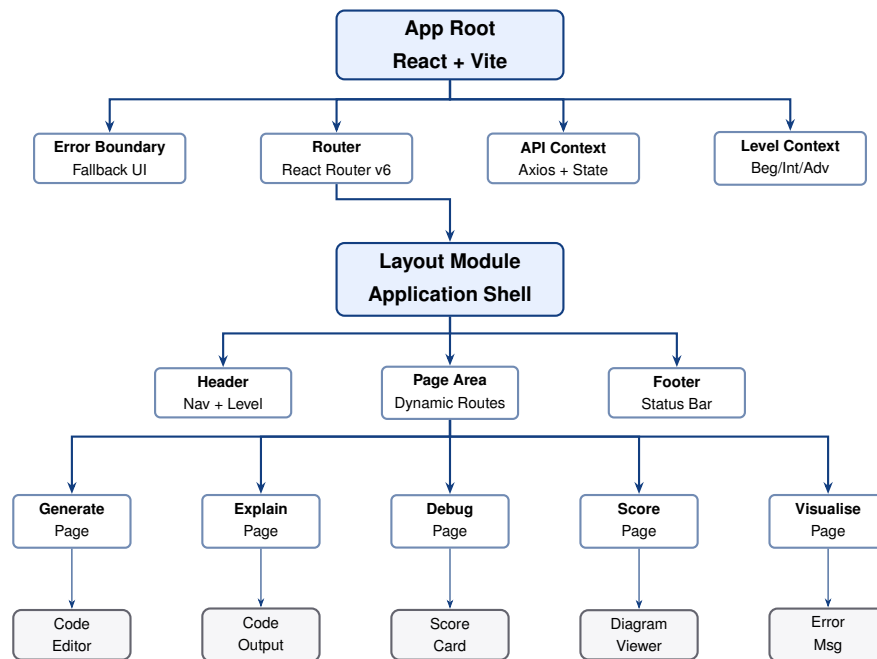


Figure 3: React Frontend Component Hierarchy

3.1.1 Architectural Layers

1. **Infrastructure Layer:** Mounts Error Boundary, React Router v6, API Context (Axios state, loading, errors), and Level Context (Beg/Int/Adv) at the application root.
2. **Layout Layer:** Provides the persistent shell with a level-badge header, dynamic page content, and status footer.
3. **Feature Layer:** Five dedicated pages (Generate, Explain, Debug, Score, Visualise) each read the Level Context and include it in every API request payload.
4. **UI Component Layer:** Shared atomic components (CodeEditor, CodeOutput, ScoreCard, DiagramViewer, ErrorMessage) receive all data via props with no business logic.

3.2 UI/UX Design Strategy

3.2.1 Design System Principles

- **Knowledge-Level First:** The level selector is prominently placed in the header; a persistent badge shows the active setting on every page.
- **Tailwind CSS Component System:** A consistent token set — blue for interactive elements, green for success, amber for warnings, red for errors — applied via Tailwind configuration.
- **Structured Output Layout:** The output panel has independent tabs for Code, Explanation, Score, and Diagram, each populated as its operation completes.
- **Responsive and Accessible:** Tailwind responsive breakpoints ensure correct display on desktop, tablet, and mobile; semantic HTML and ARIA labels support screen

readers.

3.2.2 User Experience Patterns

- **Level-Aware Single Interface:** All seven operations share a unified interface; the knowledge level is injected silently into each request without user re-configuration.
- **Progressive Disclosure:** Advanced options (target language, diagram type) appear via collapsible panels only after the primary action is initiated.
- **Score Visualisation:** The optimality score renders as a circular gauge coloured green (80–100), amber (50–79), or red (0–49) with a ranked suggestions list beneath.
- **Persistent Input Across Tabs:** The code editor retains content when switching between Explain, Debug, Score, and Visualise tabs to avoid re-entry.

4 Backend Architecture

4.1 Service Architecture

The backend follows a modular service-oriented architecture on FastAPI with dedicated routers per operation, a shared Validation Service, and direct integration with the Hugging Face runtime, Score Engine, and Mermaid/Kroki pipeline.

4.1.1 Service Components

1. **GenRouter** (POST /generate): Injects the knowledge level into a prompt template and forwards to LLMClient; parses the response to extract clean code blocks.
2. **ExplainRouter** (POST /explain): Constructs a multi-part level-aware prompt for line-by-line, function-summary, and concept-mapping explanations.
3. **DebugRouter** (POST /debug): Requests error identification, root cause analysis, and a corrected code version; parses the response into an issues list and fixed code.
4. **ScoreRouter** (POST /score): Invokes the ScoreEngine for AST analysis (Python) or LLM quality assessment (other languages), returning a 0–100 JSON ScoreReport.
5. **DiagRouter** (POST /visualise): Passes code context to MermaidBuilder, validates the syntax locally, then submits to KrokiClient; falls back to raw syntax on error.
6. **ValidationSvc:** Shared middleware on all routes; enforces non-empty input, length caps, valid level enum, and injection-pattern blocking. Returns HTTP 400 on failure.

4.2 Data Model Design

4.2.1 Core Entities

- **Session:** Stores knowledge level, ordered request history, and active code snapshot for cross-tab persistence.

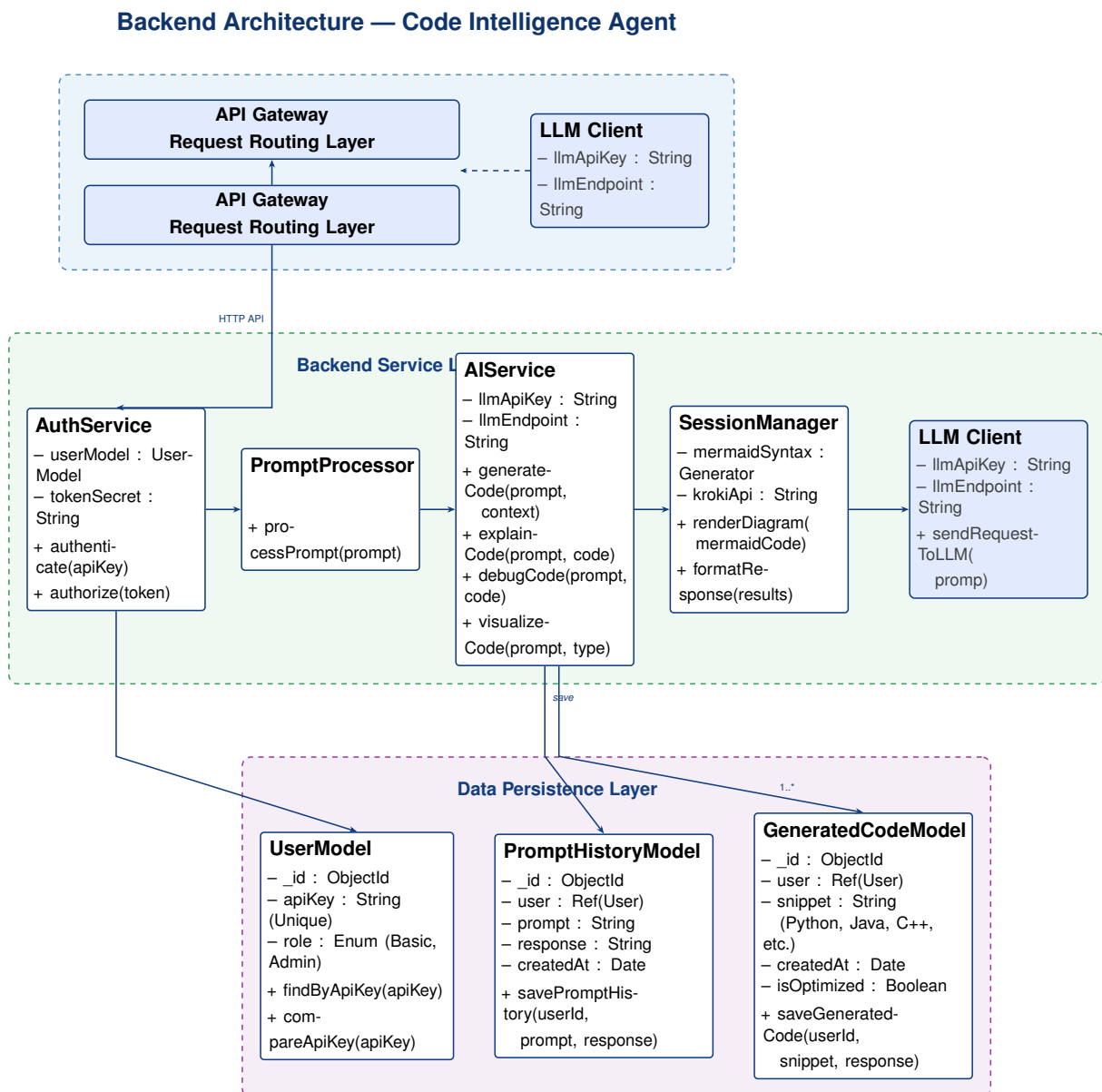


Figure 4: Backend Architecture — Code Intelligence Agent

- **Request:** Captures input type, content, operation, target language, and active knowledge level at submission time.
- **Response:** Stores generated code, explanation, debugging report, ScoreReport JSON, and rendered diagram URI linked to its originating request.
- **ScoreReport:** Captures numeric score (0–100), severity-ranked issues with line references, and ordered best-practice suggestions.
- **DiagramRecord:** Records Mermaid syntax and rendered SVG/PNG keyed by syntax hash for cache-hit elimination of redundant Kroki calls.

4.2.2 Relationship Patterns

- **One-to-Many (Session to Requests):** A session holds an ordered list of requests, providing context for follow-up queries and history display.
- **One-to-One (Request to Response):** Each request maps to exactly one AI-generated response for clear traceability.
- **One-to-One (Score Request to ScoreReport):** Every scoring request produces one ScoreReport containing the full metric set.
- **Knowledge-Level Context Linking:** All requests in a session share the same level unless the user explicitly changes it mid-session.

5 Agentic Pipeline Architecture

5.1 Seven-Stage Pipeline Strategy

The Code Intelligence Agent employs a strict sequential agentic pipeline. Each stage has a single responsibility, passes its output to the next stage, and handles failures independently with structured error responses.

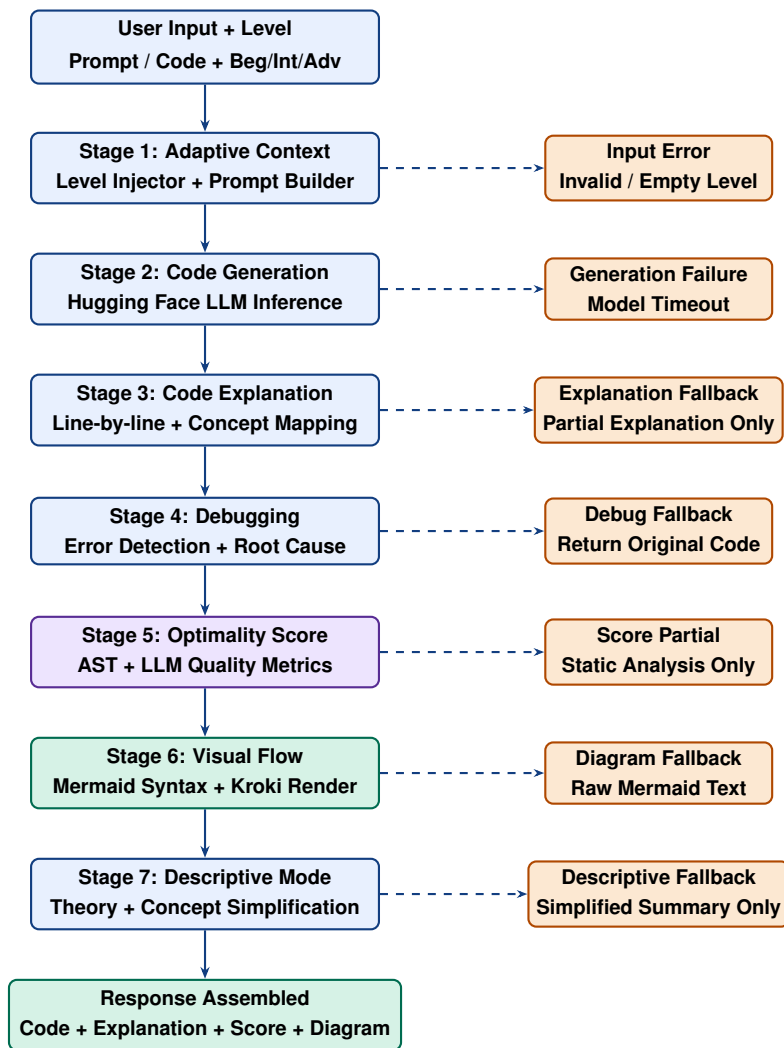


Figure 5: Seven-Stage Agentic Pipeline with Explicit Error and Fallback Paths

5.1.1 Pipeline Stage Details

1. **Stage 1 — Adaptive Context Injection:** Reads the knowledge level and constructs a stage-specific system prompt prefix setting explanation depth. Failure: returns “Invalid input or level” immediately.
2. **Stage 2 — Code Generation:** Submits the level-enriched prompt to the Hugging Face LLM; parses the returned code block. Failure: returns “Generation failed — model timeout”.
3. **Stage 3 — Code Explanation:** Produces a three-part explanation (line-by-line, function summary, concept mapping) calibrated to the level. Failure returns gracefully without halting later stages.
4. **Stage 4 — Debugging:** Identifies errors, root causes, and produces a corrected code version. Response is parsed into an issues list and fixed code block.
5. **Stage 5 — Optimality Score Check:** ScoreEngine runs AST analysis (Python) or LLM-based static assessment (other languages), returning a 0–100 JSON ScoreRe-

port. Failure returns partial static results.

6. **Stage 6 — Visual Flow Explanation:** MermaidBuilder constructs a flowchart, sequence, or class diagram; after local validation it is submitted to Kroki. Falls back to raw Mermaid syntax text on API error.
7. **Stage 7 — Descriptive Explanation Mode:** Generates a high-level, plain-language theory explanation calibrated to the user's level; no code input is required for this stage.

5.1.2 Pipeline Reliability Strategy

- **Stage-Gated Execution:** Each stage is invoked only after the previous completes; a failure halts downstream stages to avoid wasted model calls.
- **Independent Stage Failures:** Explanation, debugging, and scoring failures do not block each other; partial results are returned over total failure.
- **Timeout Guards:** LLM inference calls are bounded by `asyncio.wait_for()` with a configurable 30-second limit; expiry returns HTTP 408.
- **Stateless Execution:** Each pipeline run is fully self-contained with no shared mutable state between invocations.

6 Request–Response Communication Architecture

6.1 HTTP Pipeline Communication Strategy

The system uses a synchronous HTTP request–response model over FastAPI async endpoints. Each user operation triggers a complete, self-contained pipeline cycle: the React frontend submits an Axios POST, FastAPI orchestrates all relevant pipeline stages, and a fully assembled JSON response is returned in a single reply.

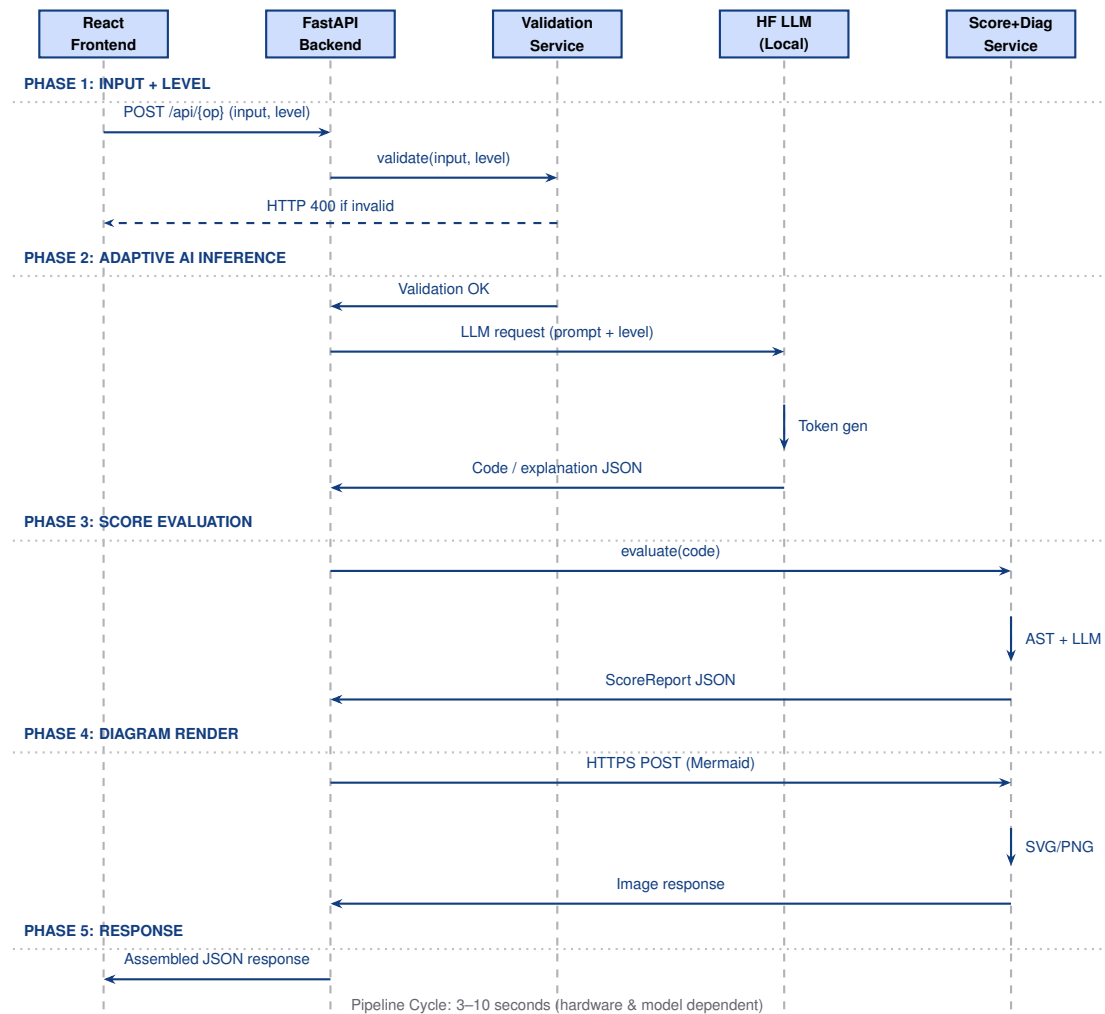


Figure 6: Request–Response Communication Architecture

6.1.1 Communication Patterns

1. **Asynchronous HTTP POST:** React submits JSON POST to the appropriate FastAPI endpoint; async handling allows concurrent sessions without blocking.
2. **Level-Enriched LLM Requests:** The `level` field is injected into every Hugging Face call, calibrating the system prompt to Beginner, Intermediate, or Advanced depth.
3. **Score Engine Invocation:** ScoreRouter calls ScoreEngine synchronously after AI inference; Python `ast` module for Python code, LLM assessment for other languages.
4. **Kroki HTTPS POST:** Diagram syntax is submitted to `https://kroki.io` as JSON over TLS; only Mermaid syntax is sent — no user code or session data.
5. **In-Memory Session Append:** Each successful pipeline run appends the record to the in-memory session store; no disk writes; data discarded on session end.

6.1.2 Event Management

- **Request Isolation:** Every POST starts a fresh pipeline instance with no shared mutable state from prior requests.
- **Timeout Enforcement:** LLM calls wrapped in `asyncio.wait_for()` with a configurable 30-second limit; expiry returns HTTP 408 with structured error body.
- **Rate Limiting:** Middleware enforces 60 requests/hour per IP, returning HTTP 429 with a `Retry-After` header.
- **Error Short-Circuit:** Pipeline stage failure immediately returns a structured JSON error without exposing stack traces.

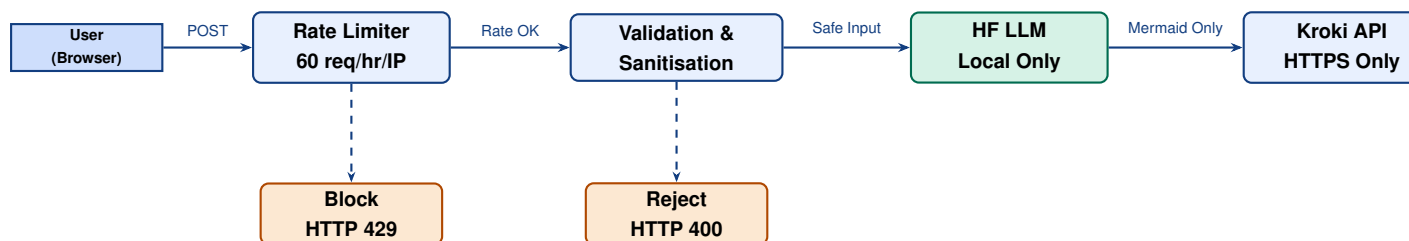
6.2 Consistency and Reliability

- **Level-First Consistency:** Knowledge level is validated before AI inference begins, guaranteeing consistent depth across all generated content for a given request.
- **Stateless Pipeline Execution:** No cross-request computation state; every run is fully reproducible from its own validated input.
- **Fallback Responses:** Kroki unavailability returns raw Mermaid syntax; LLM timeout on explanation still returns generated code output.
- **Double Sanitisation Gate:** Inputs are sanitised at FastAPI middleware and again inside `ValidationSvc` before AI processing.

7 Security Architecture

7.1 Security Model

The system implements a layered security model covering safe input handling, fully local AI processing, HTTPS-enforced external communication, and a strict no-execution policy for all user-submitted code.



Security Contract: Local AI (no data egress) + HTTPS to Kroki (diagram syntax only) + No Code Execution on Server

Figure 7: Security Architecture Overview

7.1.1 Security Strategy

1. **Input Sanitisation at Entry:** Prompts (max 1000 chars) and code (max 5000 chars) are stripped of control characters and scanned for injection patterns before AI processing.
2. **Strict No-Execution Policy:** User code is treated exclusively as text; it is never compiled, interpreted, or executed on the server, eliminating RCE vulnerabilities.
3. **Fully Local AI Inference:** All Hugging Face processing occurs within the local server process; no user data leaves the system to external cloud AI providers.
4. **HTTPS for Kroki Communication:** The only external call submits Mermaid syntax (not user code) to Kroki over HTTPS with TLS certificate validation.

7.1.2 Security Measures

- **Input Length Cap:** Oversized inputs are rejected with HTTP 400 before reaching the AI model, preventing resource-exhaustion denial-of-service.
- **HTTPS Enforcement:** Nginx reverse proxy terminates HTTPS with a valid TLS certificate; plain HTTP requests are permanently redirected.
- **API Key Management:** All credentials are stored in environment variables via `python-dotenv`; no secrets are hardcoded or version-controlled.
- **Error Masking:** Stack traces and internal paths are logged server-side only; the frontend receives a structured `{stage, message}` error object.
- **No PII Storage:** No user prompts or code are written to disk; session data is held in server memory only and discarded on session expiry.

8 Key Feature Workflows

8.1 Code Intelligence Workflow

The primary workflow covers the Generate, Explain, Debug, and Score operations through a unified client-server interaction loop with adaptive knowledge-level calibration.

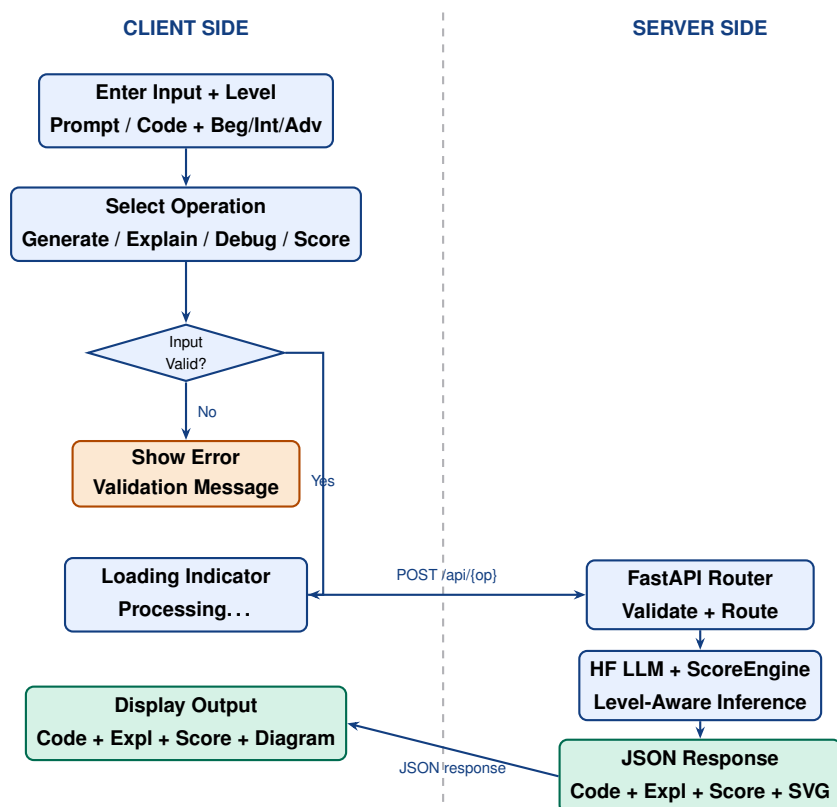


Figure 8: Code Intelligence Workflow — Client / Server Interaction

8.1.1 User Interaction Flow

- Input and Level Selection:** The user selects a knowledge level from the header badge and enters a prompt or pastes source code; both fields persist across tab switches.
- Operation Selection:** A clearly labelled button (Generate, Explain, Debug, Score, Visualise) triggers the corresponding FastAPI endpoint with level auto-attached.
- Client-Side Validation:** The frontend validates non-empty content and language selection inline within milliseconds, without a server round-trip.
- Loading State:** The button transitions to a Tailwind spinner showing the current stage label; inputs are disabled to prevent duplicate submissions.
- Output Display:** The output panel populates Code, Explanation, Score gauge, and Diagram tabs as each response section arrives.

8.1.2 Technical Implementation Approach

- Structured Prompt Templates with Level Injection:** Each operation uses a dedicated template with a common system prefix setting explanation depth for the declared level.
- Optimality Score Computation:** Python ast module for Python inputs; LLM rubric-based assessment for other languages; both produce a normalised 0–100 score.

- **Response Parsing:** Raw LLM responses are parsed with regex patterns to separate code blocks, issue lists, and Mermaid syntax into distinct Pydantic-validated fields.
- **Retry Mechanism:** A “Retry” button on error states resubmits the most recent request identically, handling transient model timeouts without user re-entry.

8.2 AI Voice / Text Integration

8.2.1 AI Integration Strategy

- **Prompt Engineering:** Carefully crafted system prompts for each stage ensure consistent, structured LLM outputs regardless of input length or phrasing.
- **Model Configurability:** The Hugging Face model identifier is stored in an environment variable and swappable without code changes.
- **Context Injection:** For follow-up requests, the most recent prior prompt-response pair is prepended to the new prompt for multi-turn coherence.
- **Output Validation:** All parsed LLM outputs are validated against Pydantic models before inclusion in the API response, preventing malformed data reaching the frontend.

9 System Integration

9.1 External Service Integration

- **Hugging Face Transformers (Local):** The transformers library loads the LLM via pipeline() at startup; model name, token limit, and sampling parameters are controlled through environment variables. No external HF API calls are made.
- **Kroki Diagram Rendering API:** The public instance at <https://kroki.io> accepts Mermaid syntax as a JSON POST and returns rendered SVG. No API key is required for basic rendering; Kroki was selected for broad Mermaid support and reliable availability.
- **Mermaid Syntax Standard:** Mermaid serves as the decoupled intermediate representation between AI-generated diagram structure and the Kroki rendering engine, enabling future renderer replacement without changing diagram generation logic.

9.2 API Design Principles

- **FastAPI RESTful Endpoints:** Resource-oriented paths with Pydantic request/response models provide automatic validation and live OpenAPI docs at /docs.
- **Structured JSON Payloads:** Requests include input, operation, language, level, and sessionId. Responses include code, explanation, score, diagram, error, and status.

- **Graceful Degradation:** API failures return user-friendly error pages with stage and message fields; no raw exceptions are exposed.

9.3 Deployment Architecture

- **Local Development:** Vite dev server (port 3000) and FastAPI/Uvicorn (port 8000) run as separate processes; a `.env` file controls model path, Kroki endpoint, and CORS.
- **Docker Compose Deployment:** Three services — frontend, backend, and a shared volume for HF model weights — with port mapping and inter-container networking.
- **Nginx Reverse Proxy:** Terminates HTTPS (Let's Encrypt TLS), routes `/api/*` to FastAPI, and serves the React static build for all other paths.
- **Cloud Readiness:** Stateless API design and Docker containerisation support deployment to AWS ECS, Google Cloud Run, or Azure Container Apps with a persistent model volume.

10 Performance and Scalability

10.1 Performance Characteristics

- **AI Inference Response Time:** Code generation and explanation target 3–8 seconds on machines with 16 GB RAM; model is resident at startup, eliminating 10–30 s load overhead.
- **Optimality Score Latency:** Python AST scoring completes under 200 ms; LLM-based scoring for other languages adds 2–4 seconds.
- **Diagram Rendering Latency:** Kroki rendering adds 1–3 seconds; the Mermaid-hash diagram cache returns cached SVG in under 5 ms on cache hit.
- **Frontend Responsiveness:** Tailwind renders with zero client-side computation overhead; output tabs populate immediately on response arrival with no layout shift.
- **Concurrent Users:** FastAPI's async event loop supports at least 20 simultaneous users on a single server, limited by RAM available for parallel LLM inference.

10.2 Scalability Strategy

- **Stateless Pipeline:** Each request is fully self-contained; horizontal scaling behind a load balancer is straightforward with session state in a shared Redis instance.
- **Local Model Option:** The `pipeline()` model identifier is swappable via environment variable; no service logic changes are required to upgrade the LLM.
- **Rate Limiting:** 60 requests/hour per IP prevents overload during evaluation; ad-

justable for production via FastAPI middleware configuration.

- **Lightweight Stack:** No database, no cache server beyond optional Redis for diagram cache — minimal infrastructure for rapid academic deployment.

11 Conclusion

11.1 Architectural Strengths

- **Clear Seven-Stage Agentic Pipeline:** Adaptive Context → Generate → Explain → Debug → Score → Visualise → Describe is easy to understand, test, and extend; each stage owns exactly one responsibility.
- **Adaptive Intelligence:** Knowledge-level calibration across all operations is the platform's defining differentiator, serving beginners and professionals from the same codebase.
- **Modern, Proven Technology Stack:** React + Tailwind CSS, FastAPI, Hugging Face Transformers, Mermaid, and Kroki are all actively maintained with strong community support.
- **Security-First Design:** Local AI inference, no-execution policy, two-layer input sanitisation, and HTTPS-only external communication provide robust multi-layer security.
- **Optimality Score Differentiator:** Combining AST static analysis with LLM quality assessment teaches users to write efficient, readable, professionally structured code.

11.2 Implementation Readiness

The architecture provides a well-structured foundation for a three-to-four-week build. The modular FastAPI router design allows frontend, backend, AI integration, and diagram rendering to be developed and tested in parallel. Pydantic models enforce API contracts from day one; FastAPI's /docs provides a live integration test harness; Docker Compose ensures consistent environments from the first commit.

11.3 Future Considerations

- **User Accounts and Persistent History:** JWT-based authentication with a PostgreSQL database for per-user session history and progress tracking.
- **IDE Plugin Integration:** A Visual Studio Code extension using the existing REST API to surface all seven features directly within the editor.
- **Fully Offline Diagram Rendering:** Replacing Kroki with a local Mermaid CLI renderer to eliminate the only external network dependency for air-gapped institutions.
- **Extended Language Support:** Adding Go, Rust, TypeScript, and SQL by extending the language enum and prompt templates without structural pipeline changes.

The Code Intelligence Agent provides a robust, secure, privacy-preserving, and adaptively intelligent foundation for an AI-powered programming assistant. Its seven-stage pipeline — combining knowledge-level adaptation, code generation, multi-level explanation, debugging with root cause analysis, quantitative optimality scoring, visual flow rendering, and descriptive theory explanation — delivers a comprehensive tool for students, trainers, and working professionals. The FastAPI + React + Hugging Face + Kroki stack ensures the platform is production-ready, extensible, and positioned for long-term academic adoption.